



ADAMA SCIENCE AND TECHNOLOGY UNIVERSITY

School of Electrical Engineering and Computing

Department Of Computer Science and Engineering

Documentation for Probability and Random Statistics Assignment

Group Assignment - 2

No	Name	ID	Sec
1	Mohammed Sadik	UGR/30960/15	3
2	Moti Alemu	UGR/30963/15	3
3	Mekhlugat Abdulwehab	UGR/30876/15	3

Submitted to:- Mrs. Gemechu Dinkisa

Submitted date:- 27/Dec/2024G.C

Documentation for Probability and Random Statistics Assignment

Course Title: Probability and Random Statistics

Instructor: Mrs. Gemechu Dinkisa

Submission Date: 27th December 2024

Contributors: Mohammed Sadik, Moti Alemu & Mekhluqat Abdulwehab

Question 1: Wide-Sense Stationarity (WSS)

Problem Statement:

Given the random process $X(t) = At$, where A is uniformly distributed over $(-2, 3)$, determine whether the process is WSS using simulation.

Solution:

1. Key Concepts:

- A process is WSS if:
 - The mean $E[X(t)]$ is independent of time.
 - The autocovariance $CX(t_1, t_2)$ depends only on the time difference $\tau = t_2 - t_1$.

2. Methodology:

- **Random Sample Generation:** A is generated using $A = a_{\min} + (a_{\max} - a_{\min}) \cdot U$, where U is a random number in $[0, 1]$.
- **Calculations:**
 - `calculateMean` computes $E[X(t)]$ at specific time points t_1 and t_2 .
 - `calculateAutocovariance` evaluates the dependence of autocovariance on t_1 , t_2 , and τ .
- **Results:**
 - Mean and autocovariance at $t_1 = 1$, $t_2 = 3$, and $\tau = 2$ are computed and printed.

3. Implementation:

- **Code:** See `WideSenseStationary.java`.
- **Execution:**
 - Save as `WideSenseStationary.java`.
 - Compile with `javac WideSenseStationary.java`.
 - Run with `java WideSenseStationary`.

4. Expected Output:

5. Mean at t_1 : 0.5000, Mean at t_2 : 1.5000

6. Autocovariance at (t_1, t_2) : 1.2500

7. Autocovariance at (t_1, t_1) : 0.4167

8. Time difference (τ): 2

9. **Conclusion:** The process $X(t) = At$ is not WSS because both the mean and autocovariance depend on absolute time.

Question 2: Time Dependency of Random Process

Problem Statement:

Given $X(t) = \cos(\omega t + \theta)$, where ω is constant and θ is uniformly distributed in $(-\pi/2, \pi/2)$, check whether the process is time-dependent.

Solution:

1. **Key Concepts:**
 - Mean $E[X(t)]$ should not depend on t for stationarity.
 - Autocovariance $CX(t_1, t_2)$ should depend only on τ .
2. **Methodology:**
 - **Random Sample Generation:** θ is generated uniformly in $(-\pi/2, \pi/2)$.
 - **Calculations:**
 - Evaluate $E[X(t)]$ and $CX(t_1, t_2)$.
 - **Implementation:** See RandomProcessTimeDependency.java.
3. **Expected Output:**
4. Mean at t_1 : 0.0000, Mean at t_2 : 0.0000
5. Autocovariance at (t_1, t_2) : 0.2500
6. Autocovariance at (t_1, t_1) : 0.5000
7. Time difference (τ): 2
8. **Conclusion:** The process $X(t)$ is time-independent and satisfies WSS.

Question 3: Stationarity of Random Process

Problem Statement:

For $X(t) = A \cos(\omega t + \theta)$, where A and θ are random, determine if $X(t)$ is stationary.

Solution:

1. **Key Concepts:**
 - A is constant or independent of time.
 - θ is uniformly distributed in $(-\pi, \pi)$.
2. **Methodology:**
 - **Mean Calculation:** Confirm $E[X(t)]$ is independent of t .
 - **Autocovariance Calculation:** Ensure $CX(t_1, t_2)$ depends only on τ .
3. **Implementation:**
 - **Code:** See RandomProcessStationary.py.
 - **Execution:**
 - Run in Python with appropriate libraries installed.
4. **Expected Output:**
5. Mean at t_1 : 0.0000, Mean at t_2 : 0.0000
6. Autocovariance at (t_1, t_2) : 0.5000
7. Autocovariance at (t_1, t_1) : 1.0000
8. Time difference (τ): 2.00
9. **Conclusion:** The process is stationary.

Question 4: Simulation of Mobile Messages

Problem Statement:

Simulate the number of messages received on a mobile phone per day using a Poisson process.

Solution:

1. **Key Concepts:**
 - Messages per day follow a Poisson distribution with parameter $\lambda = 50$.

2. **Methodology:**
 - Use `np.random.poisson( $\lambda$ , days)` to simulate the data.
 - Visualize results using a bar chart.
3. **Implementation:**
 - **Code:** See `PoissonProcess.py`.
 - **Execution:** Run in Python.
4. **Expected Output:**
5. Simulated messages per day for 30 days: [47, 52, 48, ...]
6. Messages received on day 15: 49
 - A bar chart displaying message counts across 30 days.

General Notes

- **Documentation:** The source code and results are documented in the assignment folder or available on GitHub (links provided).
- **Tools:** Use Java and Python for code execution. Ensure necessary libraries (e.g., Matplotlib for Python) are installed.
- **Conclusion:** The assignment demonstrates practical applications of random processes and stationarity analysis.